

Design and FPGA Implementation of a Parameterizable SPI Master-Slave Controller

Suhani Agarwal 22116090
Department of Electronics and
Communication Engineering
Indian Institute of Technology,
Roorkee
Email: suhani_a@ece.iitr.ac.in

Vanni Prashar 22116103
Department of Electronics and
Communication Engineering
Indian Institute of Technology,
Roorkee
Email: vanni_p@ece.iitr.ac.in

Anupriya Kumari 22116015
Department of Electronics and
Communication Engineering
Indian Institute of Technology,
Roorkee
Email: anupriya_k@ece.iitr.ac.in

Abstract—This paper presents the design, verification and FPGA implementation of a parameterizable Serial Peripheral Interface (SPI) master-slave controller written in Verilog HDL. The master and the slave are each implemented as synthesizable, single clock-domain modules with configurable data width, clock polarity (CPOL) and clock phase (CPHA), so the same RTL can be used for any of the four standard SPI modes. The two modules are wired together through an on-chip loopback in a top-level wrapper and demonstrated on a Xilinx FPGA. Switches on the board provide the byte that the master sends, LEDs display the byte that the slave receives back as well as the response the master gets from the slave, and the SPI bus signals (SCLK, MOSI, MISO, SS_n) are brought out to a Pmod header for logic-analyzer probing. Functional correctness is established by a self-checking testbench that covers reset behavior, corner-case data patterns, walking-1 vectors, mirror patterns, and back-to-back transfers. The complete design is small, fits comfortably on an entry-level Artix-7 device, and serves as a clean reference implementation of the SPI protocol on programmable hardware.

Index Terms—SPI, Serial Peripheral Interface, FPGA, Verilog, master, slave, CPOL, CPHA, Basys 3, ZedBoard, full-duplex, finite state machine

I. INTRODUCTION

The Serial Peripheral Interface (SPI) is one of the most widely used short-distance synchronous serial buses in embedded systems. It is simple, fast, full-duplex, and only needs four wires, which is why it shows up everywhere from on-board ADCs and DACs to flash memories, sensors and small displays. Despite that ubiquity, most introductory FPGA courses treat SPI as a black box on a vendor IP catalog. The point of this project is the opposite: to design the master and the slave from scratch in Verilog, understand exactly why each piece is there, and watch the link work on real hardware.

The work described here implements two synthesizable Verilog modules, `master` and `slave`, that together form a complete SPI link. Both modules are parameterizable in data width and SPI mode (CPOL/CPHA), so the same RTL can be reused for any of the four standard modes without editing the code. A small top-level wrapper instantiates one master and one slave on the same FPGA, ties their bus signals together

internally, and exposes a switch/button/LED interface to the user so the link can be driven by hand. The bus is also routed to a Pmod header for optional probing. Functional correctness is demonstrated by a self-checking Verilog testbench that exercises more than twenty data patterns and corner cases, and the design is then synthesized for and programmed onto a Xilinx FPGA board to confirm in-system behavior.

The rest of the paper is organized as follows. Section II briefly reviews the SPI protocol and the meaning of the four standard modes. Section III describes the overall system architecture. Sections IV and V detail the master and slave controllers, respectively. Section VI explains the top-level wrapper and the hardware demo. Section VII describes the verification strategy and the testbench. Section VIII reports simulation and FPGA implementation results. Section IX concludes.

II. SPI PROTOCOL BACKGROUND

SPI is a synchronous, master-driven, full-duplex serial protocol. A single master communicates with one or more slaves over four wires:

- **SCLK** – the serial clock, generated by the master.
- **MOSI** – Master Out, Slave In. The data line driven by the master.
- **MISO** – Master In, Slave Out. The data line driven by the selected slave.
- **SS_n** – Slave select, active low. The master pulls this low to begin a transfer with a particular slave and releases it high when the transfer is done.

Because both data lines are active during a transfer, SPI is inherently full-duplex: the master and the slave exchange one bit per SCLK edge in opposite directions. In this work both endpoints transmit and sample *most-significant-bit first*, which is the most common convention.

Two parameters fully describe SPI timing: clock polarity CPOL and clock phase CPHA. CPOL sets the idle level of SCLK (0 = idle low, 1 = idle high). CPHA decides which SCLK edge data is sampled on:

- **CPHA = 0:** data is *sampled* on the *leading* edge of SCLK and *shifted out* on the *trailing* edge. The first bit

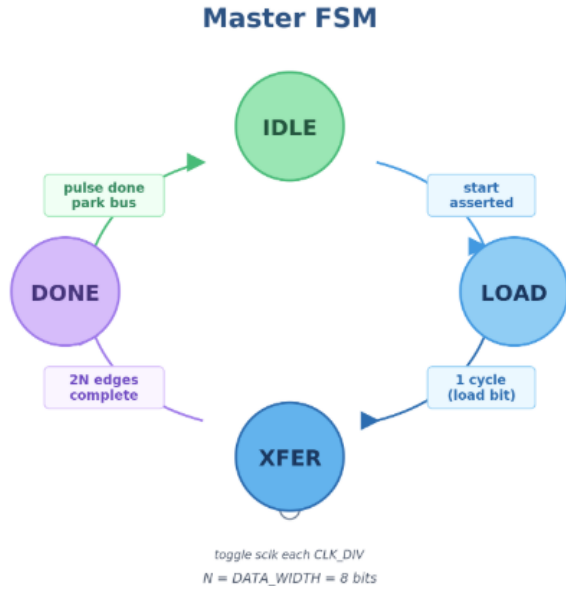


Fig. 2. Master controller state diagram

sample and the next MOSI bit-shift are then steered according to CPHA:

- For CPHA = 0, MISO is sampled on the leading edge and the next MOSI bit is shifted out on the trailing edge.
- For CPHA = 1, the next MOSI bit is shifted out on the leading edge and MISO is sampled on the trailing edge.

The final trailing edge in a word does not shift again, because there is no further bit to send. After the last edge of bit `DATA_WIDTH - 1`, the FSM jumps to `DONE`.

DONE. `SCLK` is returned to its idle level, `SS_n` is released high, the received word is latched into `rx_data`, and a one-cycle done pulse is asserted. The next clock cycle the FSM falls back into `IDLE` and the bus is again parked.

The full datapath plus FSM is implemented in a single clocked `always` block, which keeps the design easy to read and easy to synthesise.

V. SPI SLAVE CONTROLLER

The slave module is described in `slave.v`. Its job is the mirror image of the master's: while `SS_n` is low, it must sample the incoming MOSI bit on the correct edge of `SCLK`, drive the next MISO bit on the opposite edge, and after exactly `DATA_WIDTH` bits have been exchanged it must present the received word on its parallel-side `rx_data` port and pulse `rx_valid`.

A. Edge Detection, not Clock Gating

A naive slave would simply use `sclk` as its own clock and write `always @(posedge sclk)`. On an FPGA that is a bad idea: `SCLK` is not a real clock input, it is an arbitrary fabric signal, and using it as a clock leads to gated-clock warnings, poor timing closure, and unpredictable behaviour at the boundaries of a transfer.

This implementation therefore takes the opposite approach. The slave is clocked by the system `clk`, the same clock that the master uses. `SCLK`, `SS_n` and `MOSI` are treated as ordinary data inputs. `SCLK` and `SS_n` are registered once into `sclk_q` and `ss_n_q`, and the combinational expressions

$$\begin{aligned} \text{sclk_rise} &= \text{sclk} \wedge \neg \text{sclk_q} \\ \text{sclk_fall} &= \neg \text{sclk} \wedge \text{sclk_q} \end{aligned} \quad (2)$$

produce single-cycle pulses on every `SCLK` transition. From these two pulses the slave derives a *leading_edge* and a *trailing_edge* according to CPOL, and uses them as enable signals on its `posedge clk` logic. The net effect is exactly the same as “clocking on `SCLK`” but without ever actually doing so, which is much friendlier to FPGA synthesis and timing analysis.

Because in this demo the master and slave share the same system clock, a single register stage is enough for synchronisation. For an off-chip master that ran in an asynchronous domain, a second register stage in front of the edge detect would be added to handle metastability.

B. Finite-State Machine

The slave FSM has two states: `IDLE` and `XFER`, see Fig. ??.

IDLE. The slave waits for a falling edge on `SS_n`. When the master pulls `SS_n` low, the slave latches its parallel-side `tx_data` into a shift register and, for CPHA = 0, immediately drives the MSB onto MISO so that the master will see a valid bit on the very first `SCLK` edge. The FSM then moves to `XFER`.

XFER. On every leading and trailing edge pulse, the slave shifts. For CPHA = 0, MOSI is sampled on the leading edge and the next MISO bit is driven on the trailing edge; for CPHA = 1, the roles of the two edges are swapped. An edge counter, `edge_cnt`, runs from 0 to $2 \cdot \text{DATA_WIDTH} - 1$. When the final MOSI bit is sampled, the received word is presented on `rx_data` and `rx_valid` is asserted for one clock cycle. The FSM also returns to `IDLE` immediately if `SS_n` is released early.

Like the master, the entire FSM and datapath are written in a single clocked `always` block, keeping the slave compact and easy to follow.

VI. TOP-LEVEL INTEGRATION AND FPGA DEMO

The `spi_top` wrapper turns the two controllers into something a human can actually interact with on a development board. It is parameterized with the same `DATA_WIDTH`, `CLK_DIV`, `CPOL` and `CPHA` as the inner modules, plus a constant `SLAVE_TX_PATTERN` that the slave will always transmit on MISO (default `0xA5`).

A. User Interface

The board-level mapping is:

- `sw[7:0]` – byte that the master transmits on MOSI.
- `start_btn` – pressing this triggers exactly one SPI transfer.
- `rst_btn` – synchronous, active-high reset.

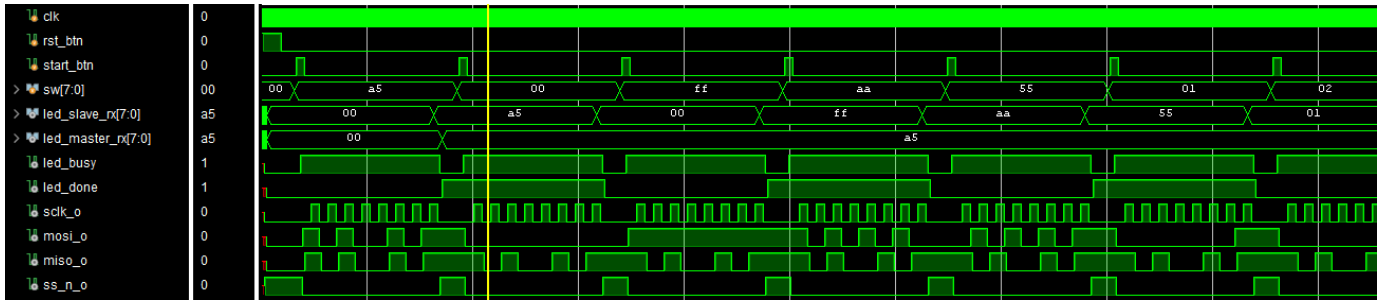


Fig. 3. Waveform of a single SPI transfer with $sw = 0xA5$. The master shifts $0xA5$ out on MOSI while the slave returns the constant pattern $0xA5$ on MISO.

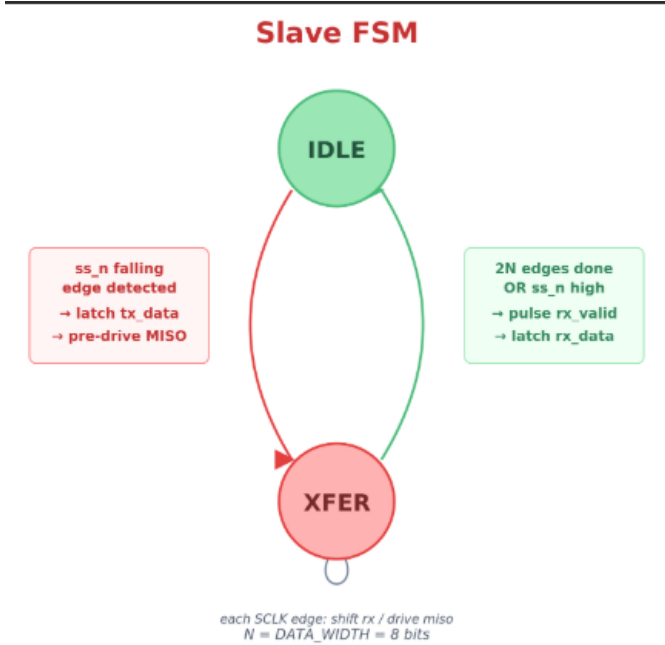


Fig. 4. Slave controller state diagram

- `led_slave_rx[7:0]` – byte the slave received (should match `sw`).
- `led_master_rx[7:0]` – byte the master received back (should be $0xA5$).
- `led_busy` – high during a transfer.
- `led_done` – toggles on every completed transfer (visible blink).
- Pmod header – carries SCLK, MOSI, MISO and `SS_n` out for scope/LA probing.

The expected user workflow is therefore straightforward: set the switches to a byte, press BTNC, and watch the bottom row of LEDs come up to match the switch pattern. The top row of LEDs (or, on ZedBoard, a Pmod 8LD module) lights up to 1010_0101, confirming that the slave $\rightarrow$ master path also works.

B. Reset Synchronisation and Button Edge Detection

Push buttons on FPGA boards are asynchronous and noisy, so `spi_top` adds two pieces of glue logic:

- A 2-flip-flop synchroniser on `rst_btn` drives a clean synchronous reset `rst` into both controllers.
- A 3-flip-flop synchroniser plus a rising-edge detector on `start_btn` produces a single-cycle `start_edge` pulse on each press.

C. Board Pin-Out

The design has been targeted at two boards, each with its own constraint file (`.xdc`). Both boards run the design at a 100 MHz system clock with `CLK_DIV = 50`, giving a 1 MHz SCLK.

- **Digilent Basys 3** (Xilinx Artix-7 XC7A35T): switches SW0..SW7 drive `sw`; BTNC is `start`, BTNU is `reset`; LD0..LD7 show `led_slave_rx` and LD8..LD15 show `led_master_rx`; Pmod JA carries the SPI bus out for probing.
- **Digilent/Avnet ZedBoard** (Xilinx Zynq XC7Z020): the on-board user LEDs LD0–LD7 show `led_slave_rx`, and `led_master_rx` is routed to Pmod JB so it can be observed on an external Pmod 8LD or a logic analyzer. SPI bus signals are routed to Pmod JA. Care is taken with the bank voltages: the DIP switches (Bank 35) and push buttons (Bank 34) use LVCMOS18 at the default ZedBoard Vadj setting.

VII. VERIFICATION

A self-checking Verilog testbench, `tb_top.v`, exercises the full `spi_top` module. The testbench drives `sw` and `start_btn` as if the user were pressing buttons on the board, waits for the busy/done handshake, and then checks both `led_slave_rx` (against the transmitted byte) and `led_master_rx` (against the constant slave pattern). A bus monitor on `ss_n_o` and `sclk_o` prints every SCLK transition during a transfer, so the waveform can be inspected from the log alone.

For simulation `CLK_DIV` is overridden to 4, which puts SCLK at 12.5 MHz and keeps each transfer short. The same DUT is then re-synthesised for the board with `CLK_DIV = 50`.

Table II summarises the test cases.

Across the 23 test cases the testbench prints a clear pass/fail line per test and a final summary. A global timeout aborts the simulation if any transfer ever stalls.

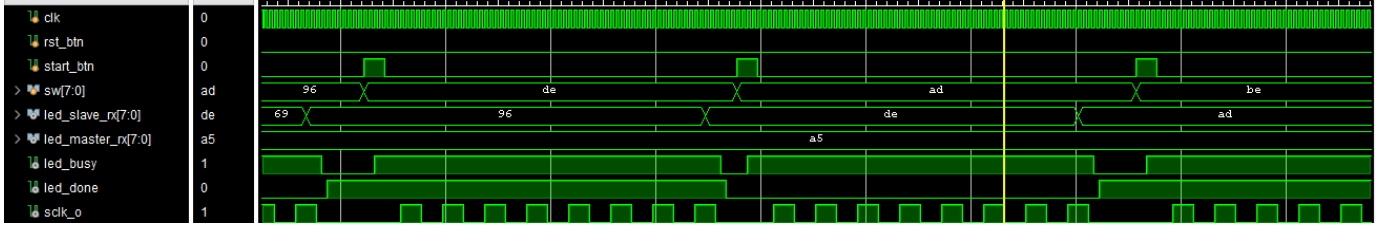


Fig. 5. Back-to-back transfers (0xDE immediately followed by 0xAD). SS_n returns high between transfers and the second start pulse correctly re-triggers the FSM.

TABLE II
SELF-CHECKING TESTBENCH COVERAGE

Group	What it checks
Reset	All outputs cleared, SS_n high, SCLK at CPOL.
Basic transfer	0xA5 round trip in both directions.
Corner zeros	0x00: MOSI must stay silent.
Corner ones	0xFF: MOSI must stay high.
Alternating	0xAA and 0x55: MSB-first ordering.
Walking-1	0x01, 0x02, ..., 0x80: every bit position.
Random patterns	0x3C, 0xC3, 0x69, 0x96.
Back-to-back	Two consecutive transfers with minimum gap.
done	led_done toggles on every completed transfer.

```

=====
TEST SUMMARY
=====
Tests run      : 21
Tests passed   : 21
Tests failed   : 0

***** ALL TESTS PASSED *****
=====
$finish called at time : 16935 ns : File "C:/Users/Suhani Agarwal/Desktop/spi_fpga

```

Fig. 6. Self-checking testbench summary in Tcl console

VIII. RESULTS

A. Simulation Results

The testbench was run in Xilinx Vivado's behavioural simulator. The console summary, the SPI bus waveform for a representative transfer, and the back-to-back transfer waveform are reproduced in Fig. ??–Fig. 5. Detailed observations are added below each figure once the screenshots are inserted; key points to call out from the waveform are:

- SS_n falls one system-clock before the first SCLK edge (the LOAD state) and rises one system-clock after the last SCLK edge.
- SCLK shows exactly $2 \cdot \text{DATA_WIDTH} = 16$ edges between each falling and rising SS_n, i.e. 8 full SCLK periods per byte.
- MOSI changes value on the trailing edge of SCLK (CPHA = 0, shift on trailing), and MISO is held valid *before* the first SCLK edge so the master can sample it on the first leading edge.
- led_busy is high for the entire duration of a transfer, and led_done toggles cleanly on each completion.

TABLE III
RESOURCE UTILISATION ON BASYS 3 (XC7A35T)

Resource	Used	Available	Util. (%)
Slice LUTs	87	20800	0.42
Slice Registers	112	41600	0.27
Slices	34	8150	0.42
Bonded IOB	33	106	31.13
BUFG / BUFGCTRL	1	32	3.13

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.768 ns	Worst Hold Slack (WHS): 0.174 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 326	Total Number of Endpoints: 326	Total Number of Endpoints: 116

All user specified timing constraints are met.

Fig. 7. Post-implementation timing report from Vivado, showing positive WNS and WHS at the 100 MHz system clock.

B. FPGA Implementation Results

The design was synthesised and implemented in Xilinx Vivado targeting the Digilent Basys 3 board (Artix-7 XC7A35T-1CPG236C). Place-and-route completed with no timing violations at the 100 MHz system clock. Table III reports the resource utilisation as extracted from the Vivado implementation report; the table is filled in once the final report is generated.

C. Timing and Power Analysis

After place-and-route, the Vivado timing and power reports were inspected to confirm that the design closes timing at the target 100 MHz system clock and to estimate its on-chip power consumption. The timing summary reports positive worst negative slack (WNS) and worst hold slack (WHS), confirming that all paths meet both setup and hold requirements with comfortable margin – unsurprising given the small size of the design and the modest target frequency. The power report breaks the total on-chip power down into static and dynamic components; the dynamic share is dominated by clocking and by toggling in the SCLK divider and shift registers, while the I/O contribution is small because only a handful of pins are driven. The two Vivado reports are reproduced in Fig. 7 and Fig. 8.

The on-board demonstration confirmed the simulated behaviour: setting the switches to a byte and pressing BTNC caused LD0 . . LD7 to update to match the switches (slave received the byte), while LD8 . . LD15 settled to 1010_0101 =

Summary

Power analysis from implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.12 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 26.4°C
Thermal Margin: 58.6°C (4.9 W)

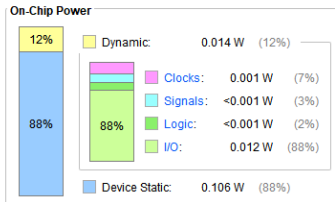


Fig. 8. Post-implementation on-chip power estimate from Vivado, broken down into static and dynamic components.

0xA5 (master received the slave's constant pattern). LD_BUSY was high for the duration of one transfer and LD_DONE toggled visibly on every press. A logic analyzer attached to the Pmod header showed a clean 1 MHz SCLK, a clean SS_n envelope, and MOSI/MISO data lines that agreed with the simulation.

IX. CONCLUSION

A parameterizable SPI master and slave have been designed in Verilog, combined in a top-level wrapper, verified with a self-checking testbench covering more than twenty data patterns and corner cases, and demonstrated on a Xilinx FPGA board. Two design choices were emphasised throughout: the whole link runs in a single clock domain (SCLK is a generated signal, not a gated clock), and the slave samples SCLK with its own clk via edge detection rather than using SCLK as a clock. Together these choices make the design synthesis-friendly and avoid the timing pitfalls that typically come with naive SPI implementations on programmable hardware.

Possible extensions include support for multiple slave selects, an AXI- or Wishbone-mapped register interface so a CPU can drive transfers in software, DMA-style streaming of larger payloads through a small FIFO, and runtime configuration of CPOL/CPHA. The clean, single-clock-domain foundation built here should make any of those extensions straightforward.

ACKNOWLEDGMENT

We would like to express their sincere gratitude to Prof. Bishnu Prasad Das for his invaluable guidance, support and encouragement throughout this project. The authors also thank the Department of Electronics and Communication Engineering, IIT Roorkee, for providing the development tools and FPGA hardware used in this work.

REFERENCES

- [1] Motorola, Inc., *SPI Block Guide V03.06*, Motorola Semiconductor Products, 2003.
- [2] F. Leens, "An introduction to I²C and SPI protocols," *IEEE Instrumentation & Measurement Magazine*, vol. 12, no. 1, pp. 8–13, Feb. 2009.
- [3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall PTR, 2003.
- [4] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ: Wiley-Interscience, 2008.
- [5] Xilinx, Inc., *Vivado Design Suite User Guide: Synthesis (UG901)*, San Jose, CA, 2023.

- [6] Digilent, Inc., *Basys 3 FPGA Board Reference Manual*, Pullman, WA, 2022.
- [7] Avnet, Inc., *ZedBoard Hardware User's Guide*, Phoenix, AZ, 2014.